INTENT BEFORE ACTION: DECODABLE AND STEERABLE SUB-GOAL REPRESENTATIONS IN LLMS

Anonymous authors
Paper under double-blind review

ABSTRACT

When an LLM writes code to satisfy a multi-step specification, does it know *what it intends to do next* before it starts writing, or is it improvising token by token? We show that code LLMs form an explicit representation of their next instrumental sub-goal *before* they emit a single token of its implementation, and that this representation is **decodable**, **steerable**, and **portable**. First, using GOALSCOPE—a lightweight probe that verbalizes the sub-goal in natural language directly from model internals at the *pivot* (the token where the model is about to begin a new sub-task)—we recover a faithful description of the upcoming sub-task well before it is written, indicating that the model plans its next step in language. Second, we localize this signal to a compact set of attention heads (*goal heads*): the goal is highly readable from these heads and essentially unreadable from the raw residual stream at the same positions, so the head selection, not the location, carries the signal. Third, the encoding is *abstract*: transplanting it steers the model to pursue the intended sub-goal in a fresh context, across programming languages, across model families and training regimes (scale, code-tuning, RL), and even in non-code sequential-planning settings. Finally, we characterize the underlying mechanism: the sub-goal is not computed at the pivot as a running difference between the specification and the model's progress; instead it is *committed upstream*—linearly present in the residual stream before the goal-head layers concentrate—and the goal heads *read out and route* a pre-decided, redundantly-stored goal (ablating them is insufficient to remove it). Our results give a concrete, testable account of how LLMs represent and pursue intermediate goals during long-horizon generation.

1 INTRODUCTION

Consider a programmer asked to write a routine that *retrieves a record from a database*, a task that decomposes into several sub-tasks: *check that the query is valid*, *check that the user has permission*, *perform the retrieval*, and so on. The programmer does not decide what to type one keystroke at a time. At each point they hold a coherent *goal* for the current sub-task (*"check that the query is valid"*) before writing any of it. We ask whether large language models (LLMs) write code the same way: when an LLM emits a long span of tokens to satisfy a multi-step specification, does it hold an explicit representation of the *next* instrumental sub-goal before it begins to implement it?

Although LLMs are trained to predict one token at a time, a growing body of work shows that they *look ahead*: they encode information about tokens they have not yet emitted (Pal et al., 2023) and plan toward constraints several tokens in the future (Lindsey et al., 2025). Most of this work treats *"the plan"* as a single, static object. We take a more mechanistic view and ask what the plan for the *next step* looks like as a concrete internal object—and, crucially, whether that object is one we can read, steer, and move between contexts.

Three-level goal hierarchy. We study the *instrumental* level of the following hierarchy: the *broader goal* (implement the function correctly) decomposes into *instrumental sub-goals* (*check the timeout is positive*), each of which admits many *implementations* (`timeout > 0` vs. `timeout >= 1` vs. `int(timeout) > 0`). Our claim concerns a representation of *which instrumental sub-goal the*

model is about to pursue, invariant to how it will be realized in code—and separable from whether it will be executed correctly.

Contributions. We give evidence, on `Qwen2.5-Coder-32B-Instruct` and replicated across model families, that the next instrumental sub-goal is an explicit, decodable, and steerable internal object, present before execution. Our findings are organized as follows.

The next sub-goal is encoded before execution, and is verbalizable (Section 3). Before the model emits a single token of a sub-task, we can read out a faithful natural-language description of what it is about to implement. We introduce GOALSCOPE, a lightweight probe built on Patchscope (Ghandeharioun et al., 2024) that verbalizes the sub-goal directly from model internals at the *pivot*. GOALSCOPE recovers the correct sub-task well above chance before implementation begins, indicating that LLMs plan their next step in language.

A sparse set of *goal heads* carries the goal (Section 4). We localize the signal to a compact set of attention heads. The goal is highly readable from these heads and essentially *unreadable* from the raw residual stream at the same positions and depths—the head *selection*, not merely the location, is what carries the goal—and a handful of heads suffice.

The encoding is abstract, steerable, and portable (Section 5). Transplanting the encoding causally steers a fresh context to pursue the intended sub-goal. This works *across programming languages* (a goal cached from Python triggers the same sub-goal injected into C++, and vice versa), *across model families and training regimes* (scale, code-tuning, RL), and *beyond code*, in few-shot and non-code sequential-planning settings—evidence that the encoding is a genuinely abstract representation of intent, decoupled from the surface code.

The goal is committed upstream, not diffed at the pivot (Section 6). We characterize where the goal comes from. It is *not* computed at the pivot as a running difference between the specification and the model's progress; it is linearly present in the residual stream *before* the goal-head layers concentrate, and the goal heads read out and route a pre-decided, redundantly-stored goal—ablating them alone does not remove it. This section also isolates the representation of *which* sub-goal is pursued from a mere *competence*/success signal.

Together, our results give a concrete, testable account of how code LLMs represent and pursue intermediate goals: an explicit, portable, causally manipulable encoding of *intent* that precedes action. In the main text we report results on `Qwen2.5-Coder-32B-Instruct`; the appendices replicate the findings across programming languages (Python and C++) and additional model families (OLMo, Appendices H and I).

arnab: Teaser figure: the read-out + one steering example, framed as "intent before action".

2 TASK, BACKGROUND, AND NOTATIONS

2.1 LANGUAGE MODELS

An autoregressive transformer language model $M : \mathcal{X} \to \mathcal{Y}$ over a vocabulary $\mathcal{V}$ maps a sequence of tokens $x = (x_1, x_2, \ldots, x_n)$, $x_i \in \mathcal{V}$, to $y \in \mathbb{R}^{|\mathcal{V}|}$, a probability distribution over the next-token continuation of x. Internally, M has L layers; the residual stream at layer ℓ and position t is $h_t^\ell = h_t^{\ell-1} + m_t^\ell + \sum_j a_t^{\ell j}$, where m_t^ℓ is the output of the MLP and $a_t^{\ell j}$ is the contribution of the j-th attention head. Following Elhage et al. (2021), the contribution of a single head to h_t^ℓ is

$$a_t^{\ell j} = \left(\sum_{i \le t} \mathrm{Attn}(q_t, K)_i \, v_i \right) W_O^{\ell j}, \qquad \mathrm{Attn}(q_t, K) = \mathrm{softmax}\left(\frac{q_t K^\top}{\sqrt{d_{\text{head}}}} \right), \tag{1}$$

with query $q_t = h_t^{\ell-1} W_Q^{\ell j}$, keys $K = h_{\le t}^{\ell-1} W_K^{\ell j}$, affinity of query and key states determine the mixture over the value states $v_i = v_i^{\ell-1} W_V^{\ell j}$, which is then projected with the output projection $W_O^{\ell j}$ and added to the residual stream. We write $[\ell, j]$ for the j-th attention head at layer ℓ, and $h_t^\ell(x)$ for the layer-ℓ residual stream at position t when M is run on x.[1]

[1] Some model families (e.g. OLMo, Gemma-2) apply RMSNorm to h before Eq. 1;

2.2 TASK: SEQUENTIAL SUB-TASK IMPLEMENTATION

We study a set of controlled settings in which the model must satisfy a *broader goal* that decomposes into an ordered set of independent *sub-tasks* $G = \{g_1, \ldots, g_K\}$, each specified in natural language (for example, *"check that the timeout is positive"*). A prompt p presents the specification and asks the model to implement the sub-tasks in code, one after another. The model realizes each g_k over a contiguous span of generated tokens. A concrete example of the task is shown on Figure **??**, with more details on Appendix A.

We use two terms throughout. The *pivot* π_k is the token position at which the model is about to begin sub-task g_k but has emitted none of its implementation. This is the natural place to ask what it *intends* to do next. The next instrumental goal at π_k is the sub-task g_k it is about to pursue; the model's *progress* at π_k is the set of sub-tasks it has already completed (equivalently, its place in the ordered list G). Our central object of study is the internal representation of the next instrumental goal g_k present at π_k, *before* g_k is realized in code.

2.3 INTERVENTION PRIMITIVES

Activation patching (Vig et al., 2020; Mueller et al., 2026). We run M on a prompt p but overwrite a chosen set of internal activations with latents cached from a different run, and read out the effect on the output distribution. We write $M\,(p)\left[a_i^{\ell j} \leftarrow v\right]$ for the forward pass on p in which the output of head $[\ell, j]$ at position i is replaced by the vector v. When the replaced value is cached from a *source* prompt p_{src} and injected into a *destination* prompt p_{dst}, a change in $M\,(p)\,[v_{\text{orig}} \leftarrow v_{\text{rep}}]$ isolates the causal role of the patched component/encoding.

arnab: M(x) usually denotes a single forward pass. Our setup is multi-token generation. Ok?

Query, key/value, and output transplants. The head decomposition in Eq. 1 exposes three places a transplant can act, and we use all three. An *output* (OV) transplant replaces a head's contribution $a_t^{\ell j}$ (what it writes to the residual stream), and is the primitive behind GOALSCOPE and our head localization (Sections 3 and 4). A *query* transplant replaces the query $q_t^{\ell j}$ that the destination uses at π_k with the source's, changing *what the read looks for*. A *key/value* transplant replaces the keys and values $K^{\ell j}, V^{\ell j}$ at the specification tokens with the source's, changing *what the read finds*. We use these transplants both to *steer* a fresh context toward an intended sub-goal (Section 5) and to probe *where* the goal is computed (Section 6).

3 THE NEXT INSTRUMENTAL SUB-GOAL IS ENCODED—AND VERBALIZABLE—BEFORE EXECUTION

Our first question is whether the next instrumental goal g_k is actually present and readable at the *pivot* π_k, *before* any of it is written. A model that improvises token by token would only reveal its goal as it decodes; a model that decides up front would already carry a representation of *"I am about to do g_k"* at π_k. We test this by *reading out* the model's goal at the pivot and checking whether g_k appears in the read-out.

3.1 GOALSCOPE: VERBALIZING THE GOAL FROM MODEL INTERNALS

GOALSCOPE is a lightweight probe that verbalizes the model's internal goal in natural language at a single token position. It extends *Patchscope* (Ghandeharioun et al., 2024) with two distinctions: (i) instead of the residual stream, it transplants the outputs of a located set of attention heads (the *goal heads* of Section 4); and (ii) it evaluates multi-token rollouts, with the intervention re-applied at every generated token, rather than a single next-token read. We inject the head outputs cached at π_k into a neutral probing prompt and decode a description of the intended sub-task. See **??** for a schematic; full details in Appendix C.

arnab: schema of the setup

3.2 RESULTS: FAITHFULNESS AND CAUSALITY OF THE READ-OUT

We apply GOALSCOPE at the *pivot* tokens and measure two quantities; both are graded by an oracle LM (`claude-sonnet-4.6`) against the gold sub-task description (Appendix C).

Faithfulness. Does the read-out faithfully describe a non-trivial part of what the model actually implements as its next *meaningful* step (the sub-task g_k)? On the locked prefill-to-frontier corpus (all datatypes, n=253), the pivot read-out attains high faithfulness *before* the first token of g_k is generated (*goal heads* row of Table 1).

Causality. Is the read-out representation causal, or merely correlated with the output? We transplant the pivot goal encoding into an information-less code stub in a *different* programming language and test whether it drives the stub to implement a non-trivial part of g_k. High causality shows the verbalized goal is the thing the model acts on, not an epiphenomenal decoding. (We expand this into full cross-context / cross-model steering in Section 5.)

?? shows the read-out achieves high faithfulness and causality well before the model writes g_k. Qualitative read-outs on free-form code generation are in Appendix J.

arnab: pivot faithfulness+causality across pivot positions

***goal heads* show *gaze* behavior.** As with the caption-generation heads of Gandikota & Bau (2026), the *goal heads* exhibit *gaze*: at π_k their value-weighted attention concentrates on the specification tokens describing the upcoming sub-task g_k. We quantify this alignment in Appendix D. Gaze, together with the read-out, becomes a debugging tool in later sections: it lets us inspect what an intervention has changed in latent space *without generating a single token*.

3.3 Discussion

The read-out establishes that a coherent, verbalizable encoding of the next instrumental goal is present at the pivot, before the model commits to any implementation. That the encoding can be read out *in natural language* indicates that code LLMs plan their next step in language. The remaining sections ask *where* this encoding lives (Section 4), whether it is *abstract and steerable* (Section 5), and *how* it comes to be present at the pivot (Section 6).

4 A Sparse Set of *goal heads* Carries the Goal

Section 3 read the goal out of a located set of attention heads. Here we say how those *goal heads* are found, and show that the goal signal is a property of the *head selection*—not of the residual stream at those positions, and not of a large fraction of the network.

4.1 Locating the *goal heads*

We localize the *goal heads* with activation patching (Section 2.3) on a minimal-pair setup. A *source* and a *destination* prompt each ask the model to implement a single predicate ($\text{pred}_{\text{src}} \neq \text{pred}_{\text{dst}}$) on the same data type (`int` | `str` | $\cdots$), each cut off so the next token is representative of the predicate being implemented. To guarantee the located latents carry the abstract predicate rather than shallow token information, source and destination use *different programming languages*, so the source's target token is absent from the destination's language (a concrete example is in ??).

We cache what each head *writes* to the residual stream (its OV contribution $a_t^{\ell j}$, Eq. 1) from the source run, and learn a sparse binary mask over all heads that, when their outputs are patched into the destination run, maximizes the logit of the source-predicate token tok_{targ}:

$$a_{-1}^{\ell j} \leftarrow \text{mask}^{\ell j}\, a_{\text{src}}^{\ell j} + (1 - \text{mask}^{\ell j})\, a_{-1}^{\ell j}. \tag{2}$$

The learned mask selects a compact set of *goal heads* concentrated in the model's middle-to-late layers (band 25–56 for `Qwen2.5-Coder-32B`; 122 heads at threshold > 0.9). Full training details, layer/threshold sweeps, and per-model head sets are in Appendix B.

4.2 The head *selection*, not the location, carries the goal

Is the goal simply present in the residual stream at the pivot, or specifically in what the *goal heads* write? We compare GoalScope's read-out quality when the injected vector is the *goal heads*' aggregate output versus the raw residual stream (and the full attention-block output) at the same positions and depths, holding the injection mechanism fixed (norm-relative, so the comparison reads content, not magnitude; Appendix C).

source injected into GOALSCOPE	faithfulness	causality
goal heads (122, output aggregate)	**0.879**	**0.792**
full attention-block output, L40–50	0.343	0.334
residual stream, L25–56	0.031	0.020
residual stream, L40–50	0.009	0.006

Table 1: The head selection does nearly all the work. GOALSCOPE read-out on the locked prefill-to-frontier corpus (Qwen2.5-Coder-32B, n=253, α=0.1), varying only the injected source. The goal is highly readable from the *goal heads*' output, only partially from the full attention output, and essentially *unreadable* from the raw residual stream, whose norm is dominated by non-goal content.

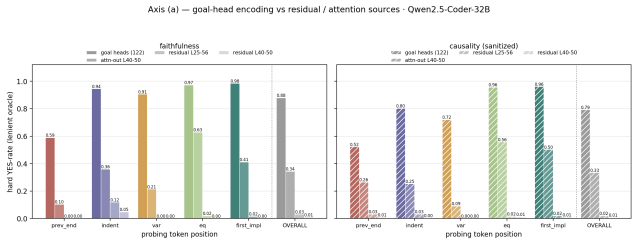


Figure 1: *goal heads* aggregate vs. raw residual / attention-block output as the GOALSCOPE source (placeholder; regenerate at final quality).

Table 1 (see also Figure 1) shows the goal is highly readable from the *goal heads* (0.88 / 0.79) but essentially *unreadable* from the raw residual at the same depths ($\sim$0.01–0.03). The head selection, not the location, is what exposes the goal.

4.3 HOW MANY HEADS? THE ENCODING IS SPARSE

The read-out saturates with only a small fraction of heads (Table 2, Figure 2). Faithfulness climbs steeply from 5 to 25 heads (0.45 $\rightarrow$ 0.79) and then plateaus: $\sim$25 heads ($\approx$0.6% of the model's 4160 heads) already recover $\sim$90% of the full-set read-out, and 50–100 heads essentially match the 122-head anchor. Adding heads beyond $\sim$100 yields no further faithfulness and a mild *causality* decline, consistent with larger sets pulling in less-selective heads that dilute the steered direction. This sparsity is what makes GOALSCOPE and the steering experiments of Section 5 a targeted intervention rather than a diffuse perturbation.

5 THE ENCODING IS ABSTRACT, STEERABLE, AND PORTABLE

Sections 3 and 4 showed the goal is decodable from a sparse set of heads. We now show it is a genuinely *abstract* representation of intent: transplanting it *steers* a fresh context to pursue the intended sub-goal, and it does so across programming languages, across model families, and even outside code. Throughout, a *source* run $M(p_{\text{src}})$ supplies the *goal heads* activations we transplant into a *destination* run $M(p_{\text{dst}})$; we report the oracle-judged **causal trigger rate** (does the destination's next computation switch to the intended sub-goal g_{targ}?) and, where applicable, the **readout-shift** (does GOALSCOPE on the patched destination now verbalize g_{targ}?) and **gaze-shift** (do the *goal heads* now look back to g_{targ}?).

5.1 TRANSPLANTING THE ENCODING STEERS THE NEXT SUB-GOAL

We construct source/destination pairs that share a task but are about to pursue *different* next sub-goals ($g_{\text{src}} \neq g_{\text{dst}}$). Transplanting the source *goal heads* states at the pivot makes the destination pursue g_{src} instead of its native g_{dst} (Figure 3). Crucially this is not a generic "perturb-and-derail": the destination continues to write correct, coherent code, but for the transplanted sub-goal. The readout-shift and gaze-shift track the causal trigger, confirming the intervention moves the intended latent, not a surface artifact.

arnab: settle final transplant variant(s) reported here; multi-predicate ordering is the cleanest "which-goal" contrast.

# heads	5	10	25	50	100	**122**	150	200	250	300
faithfulness	0.446	0.715	0.791	0.817	0.867	**0.879**	0.859	0.880	0.852	0.874
causality	0.425	0.625	0.779	0.839	0.813	**0.792**	0.816	0.794	0.750	0.779

Table 2: GOALSCOPE read-out vs. number of *goal heads* (Qwen2.5-Coder-32B, `additive_norm` α=0.1, `hard_yes` pooled over the 5 pivots, n=1265 records). The read-out saturates by ∼50–100 heads; the 122-head set used elsewhere sits on the plateau.

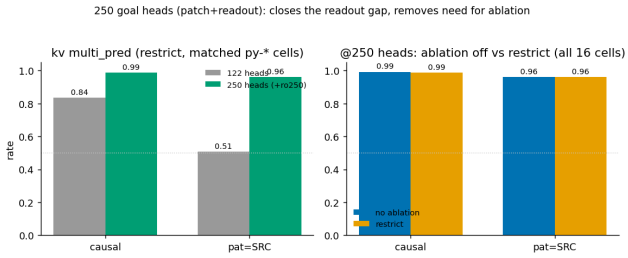


Figure 2: Read-out quality vs. number of *goal heads* (placeholder). A handful of heads recovers most of the signal.

5.2 PORTABLE ACROSS PROGRAMMING LANGUAGES

The transplant works *across the language boundary*: a goal cached from a Python context triggers the same sub-goal when injected into a C++ context, and vice versa. Because the source's target implementation is never present in the destination's language, cross-language transfer is direct evidence that the transplanted representation encodes the abstract sub-task, not its surface form. We report per-direction rates (py→cpp, cpp→py) against a same-language upper bound in Appendix H.

5.3 PORTABLE ACROSS MODEL FAMILIES AND TRAINING REGIMES

The phenomenon is not idiosyncratic to one model. We replicate the localization and GOALSCOPE read-out across (a) *scale*, (b) *code fine-tuning*, and (c) *RL post-training*, using matched head budgets per model (Figure 4; OLMo details in Appendix I). A compact set of goal heads carrying a decodable, steerable next-sub-goal encoding appears across all of them.

5.4 BEYOND CODE: FEW-SHOT AND SEQUENTIAL PLANNING

Finally, the account is not specific to code. In few-shot in-context-learning prompts, GOALSCOPE verbalizes the task the model is about to apply; and in non-code sequential-planning settings (e.g. following a recipe or an itinerary) we find qualitative read-out and steering signals of the same kind. We report these out-of-domain results in Appendix K.

arnab: quantify the ICL read-out (we said this is likely doable); keep planning as qualitative for now.

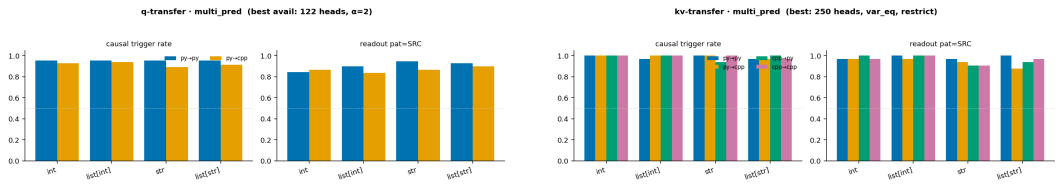
6 THE GOAL IS COMMITTED UPSTREAM, NOT DIFFED AT THE PIVOT

We have shown that a decodable, steerable goal encoding is present at the pivot. A natural mechanistic hypothesis—which we initially pursued—is that the model *computes* this goal at the pivot as a running *difference* between the task specification and its own progress (a "DIFF": $g_k \approx \textit{goal} - \textit{progress}$). This section shows that account is **not** what happens. Three results converge on a different picture: the next goal is *pre-committed upstream* and merely *read out and routed* by the *goal heads*.

6.1 THE GOAL HEADS ARE SUFFICIENT BUT NOT NECESSARY

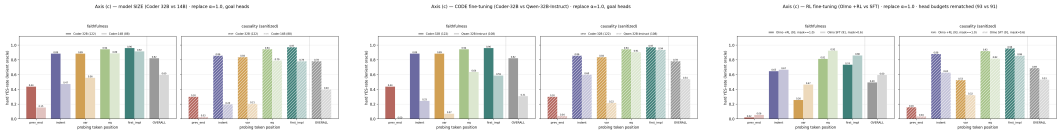
We test the *goal heads*' causal role with two matched ablations on the locked corpus (n=253; setup in Figure 5, details in Appendix C).

Sufficiency (blind-from-span). We blind *every* upper-layer head from the specification span *except* the *goal heads*. With only the *goal heads* able to read what the task asks for, the model still implements



(a) query/OV transplant (b) key/value transplant

Figure 3: Transplanting the goal encoding steers the destination to the intended sub-goal (placeholders).



(a) scale (b) code fine-tuning (c) RL post-training

Figure 4: The decodable/steerable goal encoding generalizes across model families and training regimes (placeholders).

the correct frontier sub-task (near baseline), whereas sparing a random equal-size, depth-matched set collapses to ∼0.09. The *goal heads* are *sufficient* to carry the goal into the implementation.

Necessity (sink-ablation). We instead silence the *goal heads* themselves (restricting them to attend only to the sink position from the pivot onward). This hurts *no more than* silencing a random equal-size set: the computation reroutes. The *goal heads* are *not necessary*—the goal is stored redundantly and can be recovered without them.

Sufficient-but-not-necessary means the *goal heads* are best read as a *read-out / routing channel* for a goal that is represented more diffusely, not as the unique site that *computes* it.

6.2 The goal is linearly present upstream of the goal-head layers

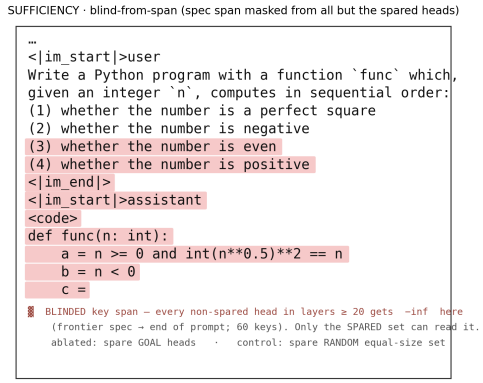
If the *goal heads* only route the goal, where is it computed? We probe the residual stream directly with **OVReader**: for a residual state h taken after layer L at the pivot, OVReader applies the *goal heads*' output–value (OV) circuit to h and reads the result through GoalScope—asking whether the upcoming sub-goal is *already present in the residual*, in the exact form a goal head would write it, from layer L onward. The goal becomes linearly readable well *before* the goal-head layers concentrate (from ∼L40; Figure 6). The *goal heads* therefore inherit a goal that upstream computation has already made available, consistent with the necessity result.

6.3 The pivot query already carries the pre-decided goal, not a difference

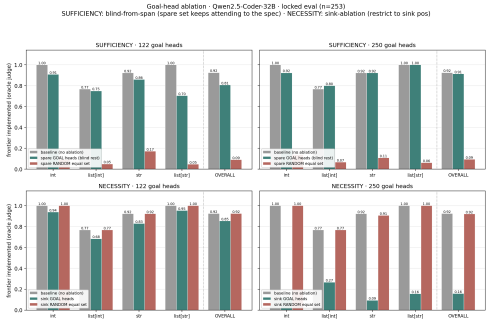
The sharpest test of the Diff account isolates *what the pivot query carries*. We hold the model's *progress identical* on both sides and diverge only the *order* of the task list, so the two hypotheses predict different next tokens:

side	task list (order)	prefilled (progress)	native next
SRC	$[T_1, T_2, \mathbf{T_3}, T_4]$	$\{T_1, T_2\}$	$\mathbf{T_3}$
DST	$[T_1, T_2, \mathbf{T_4}, T_3]$	$\{T_1, T_2\}$	$\mathbf{T_4}$

Under **(H-progress / Diff)**, the query encodes only *how far along* the model is ($\{T_1, T_2\}$, identical on both sides), and the next goal would be computed downstream from the task keys/values—so transplanting SRC's query into DST should leave DST pursuing T_4. Under **(H-committed)**, the query already carries the *computed next goal* itself, so the transplant should flip DST to T_3. We observe the latter: transplanting the pivot query flips DST toward T_3 (Figure 7). The next goal is

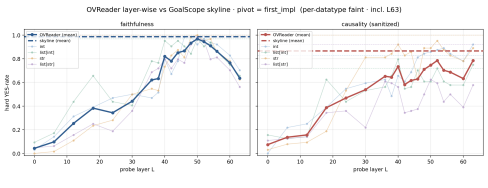


(a) sink / blind-from-span setup

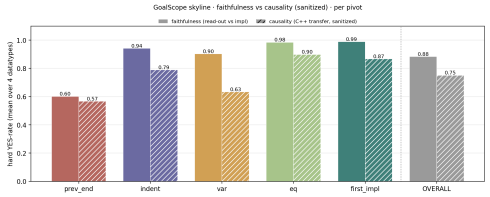


(b) sufficiency (spare goal heads) vs. necessity (silence goal heads)

Figure 5: The *goal heads* are *sufficient* (spare only them and the model still implements the sub-task) but *not necessary* (silence them and it reroutes, no worse than random). Placeholders.



(a) read-out quality vs. probe layer L



(b) OVReader vs. GOALSCOPE

Figure 6: OVReader: the goal is linearly present in the residual from ∼L40, before the goal-head band—i.e. computed upstream and read out later (placeholders).

already decided by the pivot and carried in the query; it is not (re)derived at the pivot as a difference against progress.

6.4 WHICH GOAL, NOT WHETHER IT SUCCEEDS

A direction that predicts *success* could merely track model competence rather than *which* sub-goal is pursued. Our interventions control for this: the steering and transplant tests (Sections 5 and 6.3) are *which-goal* contrasts among equally-valid orderings (T_3 vs. T_4, g_{src} vs. g_{dst}), where every option is correct, so a shift can only mean the representation encodes intent, not success. We report the failure modes—cases where the goal signal is weak or a transplant fails to trigger—in Appendix F, and discuss them next.

arnab: fold in the kv-transfer failure taxonomy (naming/datatype; patched read-out reverting to dst/neither).

6.5 SUMMARY

The next instrumental goal is *committed upstream* (linearly present in the residual by ∼L40), *stored redundantly* (silencing the goal heads reroutes), *carried pre-decided in the pivot query* (not diffed against progress), and *read out and routed* by a sparse set of *goal heads* that are sufficient but not necessary. This replaces the pivot-time DIFF picture with a commit-then-route account of goal pursuit.

7 RELATED WORKS

Planning and look-ahead in LMs. A growing body of work shows that autoregressive LMs encode information about tokens they have not yet emitted and act toward future constraints (Pal et al., 2023; Lindsey et al., 2025). We refine this picture from a single static "plan" to an explicit, decodable representation of the *next instrumental sub-goal*, present before execution, and show it is steerable and portable rather than an epiphenomenal decoding.

claude: Add citations: future-lens / prompt-injection look-ahead, poetry-planning (Anthropic biology), chess/maze planning, emergent world models.

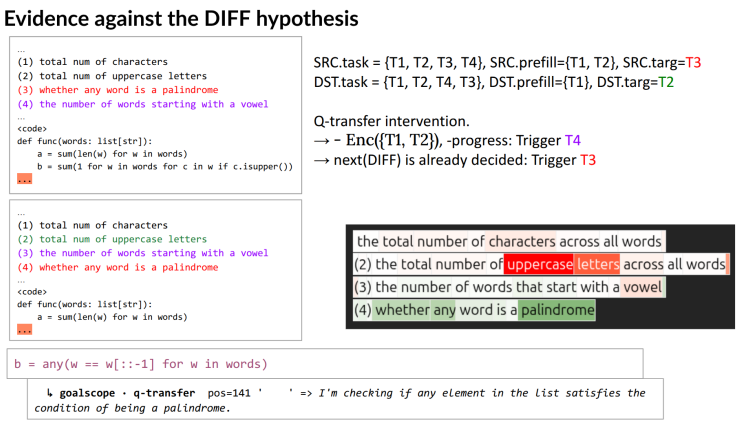


Figure 7: Query transplant with progress held fixed and task order swapped: DST flips to the source's *pre-decided* next goal T_3, not to the progress-implied T_4—evidence against a pivot-time diff (placeholder).

Portable task representations and function abstractions. Compact, transplantable representations of a task have been found in attention-head outputs (function vectors, Todd et al., 2024), in concept-induction heads that carry language-independent word meanings (Feucht et al., 2025), and in *filter heads* that encode a filtering predicate in their *query* states and reapply it across formats, languages, and tasks (Sen Sharma et al., 2026). Our *goal heads* extend this line to *intent*: a compact, transplantable, cross-language encoding of the next instrumental sub-goal, read out and routed by a sparse head set. The filter-heads finding that models sometimes *eagerly evaluate* rather than carry a portable predicate connects to our failure-mode analysis (Section 6.4).

Verbalizing latents. GOALSCOPE builds on Patchscope (Ghandeharioun et al., 2024) and related self-interpretation / lens methods, adapting them to read attention-head outputs at a single position over multi-token rollouts.

claude: Add: logit lens, tuned lens, SelfIE, function-vector readouts; contrast what GOALSCOPE reads (head-output, multi-token) vs. residual-stream single-token lenses.

High-level interpretability of goal-directed behavior. Our work is an instance of the high-level, mechanism-agnostic interpretability agenda (Greenblatt, 2024): we build a theory of goal representation at an abstraction above the token-to-syntax mapping and validate it with intervention-style experiments that transfer across programming languages, model families, and domains.

claude: Cite causal abstraction / DAS (Geiger et al.), high-level causal mediation; position our cross-language transfer as the abstraction test.

8 DISCUSSION

We gave evidence that code LLMs hold an explicit representation of their next instrumental sub-goal *before* they execute it, and that this representation is decodable (GOALSCOPE verbalizes it in natural language, Section 3), localized to a sparse set of *goal heads* (Section 4), steerable and portable across languages, model families, and domains (Section 5), and *committed upstream* rather than computed at the pivot (Section 6). Intent precedes action, and it does so as a concrete, manipulable object.

Commit-then-route, not diff-at-the-pivot. We initially modeled next-step selection as a running *diff* of the specification against progress. The evidence does not support that: the goal is linearly present in the residual before the goal-head layers concentrate, silencing the goal heads reroutes rather than removes it, and the pivot query already carries the *pre-decided* goal rather than a difference against progress. The *goal heads* are best understood as a sufficient-but-not-necessary *read-out and routing* channel for a goal that is decided upstream and stored redundantly. Where exactly the goal is first *computed* (and how the ordered task list is turned into a committed next step) is the natural next question.

Intent vs. competence. A representation that predicts success could merely track competence. Our which-goal contrasts among equally-valid orderings (Section 6.4) isolate *which* sub-goal is pursued from *whether* it will be executed correctly, but a fuller characterization of the failure modes—when the goal signal is weak, or a transplant fails to trigger—is important for trusting the account.

Limitations and outlook. Our task decompositions are specified in the prompt; whether the same representation governs *proactive* (unspecified) decomposition is only tested qualitatively (Appendix J). We also deliberately leave the low-level goal-to-syntax mapping unexamined, focusing on the abstract goal encoding. Extending the account to free-form code and to non-code sequential tasks (Section 5.4) is natural future work, as is locating the upstream computation that commits the goal.

claude: Merge with `Text/100_limitations.tex` if a separate Limitations section is desired for the venue.

ETHICS

REPRODUCIBILITY

ACKNOWLEDGEMENTS

REFERENCES

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 1(1):12, 2021.

Sheridan Feucht, Eric Todd, Byron C Wallace, and David Bau. The dual-route model of induction. In *Conference on Language Models (COLM)*, 2025. URL https://openreview.net/forum?id=bNTrKqqnG9.

Rohit Gandikota and David Bau. Gaze heads: How vlms look at what they describe. *arXiv preprint arXiv:2606.14703*, 2026.

Asma Ghandeharioun, Avi Caciularu, Adam Pearce, Lucas Dixon, and Mor Geva. Patchscopes: A unifying framework for inspecting hidden representations of language models. In *International Conference on Machine Learning (ICML)*, 2024.

Ryan Greenblatt. High-level interpretability: how LLMs make decisions in pursuit of goals, 2024. Interview remarks; high-level, intervention-style interpretability of goal-directed behavior.

Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, Joshua Batson, et al. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL https://transformer-circuits.pub/2025/attribution-graphs/biology.html.

Aaron Mueller, Jannik Brinkmann, Millicent Li, Samuel Marks, Koyena Pal, Nikhil Prakash, Can Rager, Aruna Sankaranarayanan, Arnab Sen Sharma, Jiuding Sun, et al. The quest for the right mediator: Surveying mechanistic interpretability for nlp through the lens of causal mediation analysis. *Computational Linguistics*, 52(1):331–378, 2026.

Koyena Pal, Jiuding Sun, Andrew Yuan, Byron C. Wallace, and David Bau. Future lens: Anticipating subsequent tokens from a single hidden state. In *Conference on Computational Natural Language Learning (CoNLL)*, 2023.

Arnab Sen Sharma, Giordano Rogers, Natalie Shapira, and David Bau. LLMs process lists with general filter heads. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://filter.baulab.info/. arXiv:2510.26784.

Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 12388–12401. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/92650b2e92217715fe312e6fa7b90d82-Paper.pdf.

A More Details on the Task Setup and Results

Full prompt + details.

Segmentation using an Oracle LLM.

Evaluation using an Oracle LLM.

B Localizing Attention Heads

C GoalScope

C.1 Setup

Pivot. For each sub-task g_k, we identify a *pivot* position π_k: the token at which the model has committed to implementing g_k but has not yet emitted any token of its implementation.[2] The pivot is the natural place to ask what the model *intends* to do next, before any implementation choice has been made.

Patchscope (Ghandeharioun et al., 2024). To read the *content* of a hidden representation in natural language, we transplant it into a separate *probe* prompt $\tilde{p}$ whose continuation verbalizes whatever the injected state encodes. Concretely, we replace one or more activations of $\tilde{p}$ at a chosen layer and position with a representation cached from the source, then decode $M(\tilde{p} \mid \cdot)$ greedily. We build our intent read-out, GoalScope, on this primitive (Section 3).

We introduce GoalScope, a patchscope (Section 2.3) that verbalizes the model's intent at a pivot.

Given a generation in which the model implements G, we run M once and cache, at the pivot π_k, the outputs of a set of *goal heads* $\mathcal{H}_{\text{goal}}$ (localized as described below). We then transplant these cached activations into the probe prompt

```
System: You are a helpful assistant proficient in programming.
User: What are you doing at this line of the code? Keep your answer as
brief as possible.
Assistant:
```

by replacing the corresponding head inputs to $W_O^{\ell j}$ at the probe's final token, over the layers spanned by $\mathcal{H}_{\text{goal}}$, and decode the continuation greedily. The decoded text is GoalScope's verbalization of the intent carried at π_k.

arnab: finalize: additive vs. replace strategy, layer range, α; report the one used in the main results and defer the ablation to the appendix.

Locating goal heads. We identify $\mathcal{H}_{\text{goal}}$ with a differentiable head mask trained to transfer intent across contexts: activations at the pivot of a *source* sub-task, patched into a *destination* pivot for a different sub-task, should redirect the destination toward the source's sub-task (Section 2.3). The heads selected by the mask are those whose pivot activations causally carry the identity of the upcoming sub-task.

arnab: cross-ref the localization method/appendix; state $|\mathcal{H}_{\text{goal}}|$ and layer range.

C.2 Evaluation

C.3 Baselines/Existing works

C.3.1 Function Vector

C.3.2 J-Lens

C.3.3 Activation Oracle

C.4 Discussion/Opinions

arnab: our take on the lens methods. what info in the latents we can (and probably should) expect to be verbalizable in natural language.

[2] In practice π_k is the whitespace/newline token that begins the line implementing g_k; **??** gives the exact tokenization.

D GAZE MECHANISM

D.1 SETUP

D.2 EVALUATION

The goal heads look back to what they are implementing.

The LM implements where the goal heads look back to.

D.3 GAZE HEADS VS CONCEPT-COPIER HEADS

E BOUNDARY ENCODING

E.1 APPLYING `move-on` INTERVENTION IN THE MIDDLE

F CHECKING FOR FUNCTIONAL DISSOCIATION

F.1 GOAL HEADS VS IMPULSE HEADS

G ROLE OF MLPS

arnab: TODO: path patching experiments: patch + freeze MLPs with dest values. Measure intervention effects.

H GENERALIZATION ACROSS PROGRAMMING STYLES AND LANGUAGES

I GENERALIZATION ACROSS LMS

arnab: replicate everything across at least 1 model different model size, and 1 different model family

J GENERALIZATION TO UNSPECIFIED SETTINGS

K GENERALIZATION TO NON-CODE SETTINGS